

Answers for Re-exam 2024

Written examination date: 21.08.2024

Page 1 of 8 pages

Course name: Python and High-Performance Computing

Course number: 02613

Aids allowed: Written works of reference are permitted

Exam duration: 4 hours

Weighting: The grade is determined based on an overall assessment of all answers to the exam questions.

For all *non-multiple choice* problems, when answering the different questions, **remember to show/explain your calculations and motivate your answers**. If you make additional assumptions, remember to briefly explain them. Failure to do so may cause substantial reduction of the number of points given. You may write your answers directly on the exam set, on separate papers, or both. For answers on separate papers, remember to clearly indicate what question you are answering.

For all *multiple choice* problems you may simply mark your chosen answer with no additional justification. If you wish to change your answer, cross out your current mark and write your chosen option next to the question.

Unless otherwise specified:

- All arrays are NumPy arrays and stored row-wise.
- `np` refers to the NumPy Python module.

You are a performance consultant and have been sent by DTU to help out a company called Strategic Networked Advanced Intelligence Logistics. They are doing many different kinds of data processing, all of which are currently running too slow.

One of the engineers is using batch jobs to perform some data processing. They show you a job script they are working on:

```
1  #!/bin/bash
2  #BSUB -J classify
3  #BSUB -q hpc
4  #BSUB -W 02:00
5  #BSUB -R "rusage[mem=4GB]"
6  #BSUB -n 8
7  #BSUG -R "span[hosts=1]"
8  #BSUB -o proc_%J.out
9  #BSUB -e proc_%J.err
10
11 python classify.py data.csv
```

1. What resources does this job script request? Mention maximum wall-time, memory usage and cpu cores.

The job script requests:

2 hours of wall time.

8 cores

4 GB of memory per core which is a total of 32 GB.

To improve performance, the engineers have restructured `classify.py` to run their algorithm on a GPU.

2. How would you change the job script above such that the program is run on a node with a GPU?

Change the queue to one of:

- `gpub100`
- `gpua100`
- `hpcintogpu`

Also, add the the following option:

`#BSUB -gpu "num=1:mode=exclusive_process"`

Another engineer is working on a different program using CPU parallel processing. The program takes 10 minutes to run on 4 processors. They have estimated the parallel fraction as $F = 0.8$.

3. How many minutes would the program take to run on 1 processor?

Using Amdahl's law we can compute the speed-up:

$$S(4) = \frac{1}{1 - F + \frac{F}{p}} = \frac{1}{\frac{1}{5} + \frac{4/5}{4}} = 5/2 = 2.5.$$

Since the time on 4 processors was 10 minutes, the time on 1 processor must therefore be $10 \cdot 2.5 = 25$ minutes.

An engineer now claims they can reduce the run time of the non-parallelizable part by 3 minutes.

4. Given the reduced serial run time, what will the new run time on 4 processors be?

Original total time (in minutes) on 4 processors was

$$T_{\text{serial}} + \frac{T_{\text{parallel}}}{4} = 10.$$

As only T_{serial} has changed, the new time in minutes is

$$T_{\text{serial}} - 3 + \frac{T_{\text{parallel}}}{4} = 10 - 3 = 7.$$

They now send you to another department. Here, an engineer is working on an application where they have a matrix of size $1\,000 \times 100\,000$ with data type `float64`. The matrix is too large for them to load it into memory, so they will store it as a Zarr array and read it in blocks. For their application, they need to compute the sum of a given set of columns, i.e., their code is:

```
1 def process(fname, columns):
2     x = zarr.open(fname, mode='r')
3     s = 0
4     for i in columns:
5         s += x[:, i].sum()
6     return s
```

5. Only considering the above application, which of the following block shapes for the Zarr array will achieve the best performance?
- (a) $10 \times 10\,000$ (b) $100 \times 1\,000$ (c) $1\,000 \times 100$

The answer is (c).

Since zarr arrays read blocks in from disk when accessed, the best block shape is the one which requires the fewest block reads. We always read an entire column at a time and (c) is the only layout which contains an entire column per block. Therefore, (c) is best.

6. How much memory will it take to load a single block into memory?

Each block contains 100 000 elements of type `float64`. Therefore, each block will use $100\,000 \cdot 8 = 800\,000$ bytes or roughly 800 KB.

Satisfied with your help, they now send you to the simulation department. Here, they are trying to optimize a function named `simulate`. To this end, they have run it through a line profiler and gotten the following output:

Line #	Hits	Time	Per Hit	% Time	Line Contents
3					@profile
4					def simulate(x, y, n_steps):
5	1	1.0	1.0	0.0	z = 0.0
6	1	1.0	1.0	0.0	n = len(x)
7	10001	2000.0	0.2	13.3	for i in range(n_steps):
8	10000	5000.0	0.5	33.3	a = x[i] * x[i] + 4
9	10000	5000.0	0.5	33.3	b = y[n - i - 1] / x[i]
10	10000	3000.0	0.3	20.0	z = z + a / b
11	1	0.3	0.3	0.0	return z

7. How many steps was the simulation run for, i.e., what was the value of `n_steps`?

Each line in the loop was hit 10 000 times so the simulation ran for 10 000 steps.

The time unit for the above profiler output is in microseconds (μs) and $1\text{ s} = 10^6\ \mu\text{s}$. Assume `x` and `y` have type `float32`.

8. How many FLOP/s (floating point operations per second) does the above function achieve?

- (a) $2.67 \cdot 10^6$ FLOP/s (b) $3.33 \cdot 10^6$ FLOP/s (c) $4.67 \cdot 10^6$ FLOP/s

Only the operations in the for loop are floating point operations.

Computing `a` on L8 uses 2 FLOPs: the multiply and the add.

Computing `b` on L9 uses 1 FLOP: the divide.

Updating `z` on L10 uses 2 FLOPs: the add and the divide.

In total, 5 FLOPs per iteration of the loop, totaling $5 \cdot 10\,000 = 50\,000$ FLOPs.

The total time for the function was $2\,000 + 5\,000 + 5\,000 + 3\,000 = 15\,000\ \mu\text{s} = 15\text{ ms} = 0.015$ seconds.

Therefore, the functions achieves

$$\frac{50 \cdot 10^3}{15 \cdot 10^{-3}} = \frac{10}{3} \cdot 10^6 = 3.33 \cdot 10^6 \text{ FLOP/s.}$$

Instead of using a Python loop, you suggest computing the answer using NumPy array operations.

9. Which of the following NumPy expressions will compute the same value as the `simulate` function in the above profiler output?

- (a) `(x * x + 4) / (y[::-1] / x)`
(b) `np.sum((x * x + 4) / (y / x))`
(c) `np.sum((x * x + 4) / (y[::-1] / x))`

The answer is (c). It is *not* (a) because no sum is being computed. It is *not* (b) because `y` is not being reversed.

Impressed with your knowledge about NumPy, another engineer asks for your help with a NumPy broadcasting question. They have two NumPy arrays `a` and `b`. The shape of `a` is $100 \times 1 \times 6 \times 3$ and the shape of `b` is $100 \times 1 \times 3$.

10. Given the operation `c = a + b`, what is the shape of the output array `c`?
(a) $100 \times 100 \times 6 \times 3$ (b) $100 \times 1 \times 6 \times 3$ (c) Won't broadcast.

The answer is (a).

First, line up the shapes to the right:

a: 100, 1, 6, 3
b: 100, 1, 3

Then, left-pad with 1's so the shapes have the same length:

a: 100, 1, 6, 3
b: 1, 100, 1, 3

Next, replace 1's with values from the other shape:

a: 100, 100, 6, 3
b: 100, 100, 6, 3

Since all values now match the arrays will broadcast and the shape will be $100 \times 100 \times 6 \times 3$.

Next, you are sent to the CUDA department. Here, they are working on a CUDA kernel that takes an image, `x`, which is stored as an $n \times n$ array, as an input. It then computes a 3×3 average around every pixel and this result is then added to an output array `y`. Their code is:

```
1 @cuda.jit
2 def average3x3(x, y, n):
3     row, col = cuda.grid(2)
4     s = 0.0
5     if 1 <= row < n - 1 and 1 <= col < n - 1:
6         for j in range(-1, 2):
7             for i in range(-1, 2):
8                 s += x[row + i, col + j]
9         y[row, col] += s / 9 # Add to y
```

11. Which of the following thread block configurations will have the *best* performance, i.e., run the *fastest*?
- (a) 16×16 (b) 256×1 (c) 1×256

The answer is (c).

12. Explain your reasoning for the answer to the previous question:

At each iteration of the loops, every thread in a warp (could also write thread block) will access an element of x at the same time. For good performance, the threads should access sequential elements in order to be cache efficient. This only happens for (c) where threads are aligned along a row.

The engineers now want to apply this kernel to several images and add up all the results. The images are stored in an $m \times n \times n$ array x_all , where m is the number of images. Since only one $n \times n$ image can fit in GPU memory, they apply the `average3x3` kernel one image at a time using the following code:

```
1 def sumavg(x_all, threadsperblock, blockspergrid):
2     y = np.zeros(x_all.shape[1:], dtype=x_all.dtype)
3     for x in x_all:
4         average3x3[blockspergrid, threadsperblock](x, y, len(x))
5     return y
```

Assume x_all is a $100 \times 2000 \times 2000$ NumPy array in host memory.

13. How many host to device (HtoD) and device to host transfers (DtoH) will `sumavg` perform if called with x_all ?

Since both x and y are NumPy arrays in host memory, Numba will transfer both arrays to and from the GPU at every iteration. Since we have 100 images this will result in 200 HtoD transfers and 200 DtoH transfers.

14. How many transfers would be needed in an optimal implementation of `sumavg`?

The y array could be allocated directly on the GPU and would only need a single transfer back to the host after the loop. Each of the 100 images must be transferred to the GPU one at a time, but they do not need to be transferred back.

Total: 100 HtoD transfers and 1 DtoH transfer.

Happy with your analysis, an engineer from another department wants your input on a large processing task. They are processing ten large files using a job array after which a final job will compute the end result once the job array has finished successfully. They have created the following two job scripts for this purpose: They submit the jobs and after a few hours they call `bjobs -A` to check the progress. They get the following output:

<pre> 1 #!/bin/bash 2 #BSUB -J prepare[1-10] 3 #BSUB -q hpc 4 #BSUB -W 2:00 5 #BSUB -n 1 6 #BSUB -R "rusage[mem=512MB]" 7 #BSUB -o batch_output/prepare_%I_%J.out 8 #BSUB -e batch_output/prepare_%I_%J.err 9 10 11 12 python prepare.py \${LSB_JOBINDEX} </pre> bjob_prepare.sh	<pre> #!/bin/bash #BSUB -J compute #BSUB -q hpc #BSUB -W 5:00 #BSUB -n 8 #BSUB -R "span[hosts=1]" #BSUB -R "rusage[mem=512MB]" #BSUB -w done(prepare) #BSUB -o batch_output/compute_%J.out #BSUB -e batch_output/compute_%J.err python compute.py </pre> bjob_compute.sh
---	--

```

1  $ bjobs -A
2  JOBID      ARRAY_SPEC  OWNER    NJOBS PEND DONE  RUN EXIT SSUSP USUSP PSUSP
3  22185215   prepare[   user123      10    0    4    5    1    0    0    0

```

15. When will the `compute` job start?
- (a) As soon as possible
 - (b) When the remaining jobs have finished
 - (c) Never

The answer is (c).

One of the jobs has exited with failure (it is in the `EXIT` state). For an array to finish successfully, *all* of its jobs must finish successfully. Therefore, the condition for the `compute` job will never be fulfilled so it will never start.

Satisfied with your advice, the engineer sends you on to their final department. Here, another engineer is trying to sum all numbers in a large $n \times n$ matrix `x`. To speed this up, they want to use parallel processing. They get the idea to sum each row or column in parallel and then sum the row/column sums serially afterwards. However, they are unsure whether performing the parallel sum over the rows or columns is best. Concretely, they have the following two implementations:

<pre> 1 def summatrix(x, n_cores): 2 with ThreadPool(n_cores) as p: 3 # Sum rows in parallel 4 rowsums = p.map(5 lambda a: a.sum(), x 6) 7 return sum(rowsums) </pre> Parallel row sum implementation	<pre> 1 def summatrix(x, n_cores): 2 with ThreadPool(n_cores) as p: 3 # Sum columns in parallel 4 colsums = p.map(5 lambda a: a.sum(), x.T 6) 7 return sum(colsums) </pre> Parallel column sum implementation
---	---

Assume n is large enough that a row/column cannot fit in the CPU cache.

16. Which of the two parallel implementations would you expect to be faster? Explain your reasoning:

I would expect the parallel row sum to be faster. For the parallel column sum, each thread is summing over a column. This requires strided access which is not cache efficient.

This uses multi-threading. The engineer worries whether it would be better to use multi-processing instead.

17. Is it appropriate to use multi-threading instead of multi-processing in this case? Explain why/why not:

Yes, it is appropriate. Since NumPy releases the GIL and each thread is only running a NumPy function, we can use multi-threading.

Another engineer states that since you are summing all numbers, you should just reshape x to a flat array of length n^2 and then perform a parallel reduction. Assume you have unlimited computational resources available.

18. How much faster would a parallel reduction be compared to the approach from question 16? Explain your reasoning and provide your answer as a function of n :

The approach in question 16 will use time $2n$ (n for the parallel part and n again to sum the row/column sums). A parallel reduction will use time $\log_2(n^2) = 2\log_2(n)$. The speed-up with a reduction will therefore be:

$$\frac{2n}{2\log_2(n)} = \frac{n}{\log_2(n)}.$$

Note the similarity to the analysis from the slides in week 6.

End of exam